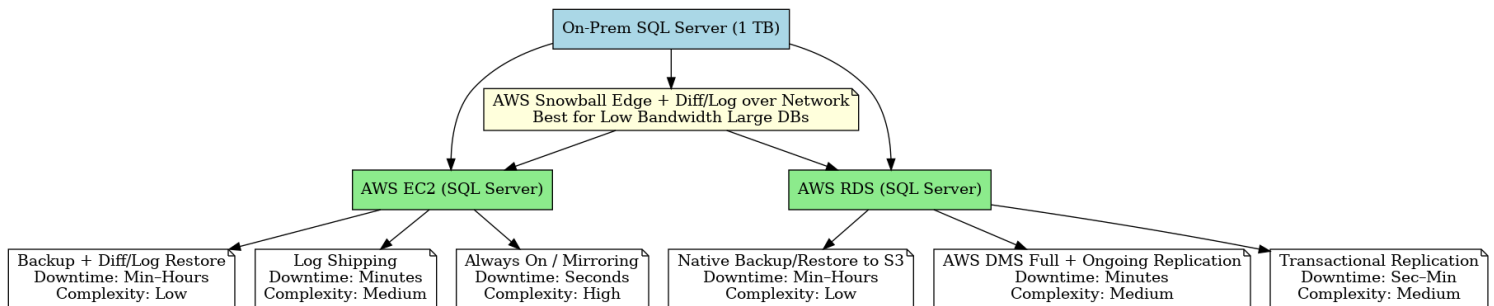


Migrate SQL Server On-prem DB to AWS EC2\RDS || Multiple Ways

You've got a **1 TB on-prem SQL Server database** and you want to migrate it to **AWS** – either to **EC2 (self-managed SQL Server)** or **RDS for SQL Server (managed service)** – with **minimal downtime**.

We'll look at **multiple migration approaches**, their pros, cons, and downtime considerations.



1. Migration to AWS EC2 (SQL Server on a VM)

Since you control the SQL Server instance in EC2, you have more flexibility.

Option A – Backup & Restore with Network Copy

Process:

1. Take a **FULL backup** on-prem.
2. Copy **.bak** to S3 (via AWS CLI, Storage Gateway, or Snowball if bandwidth is slow).
3. Restore backup in EC2 SQL Server.
4. For reduced downtime:
 - After full restore, take **DIFFERENTIAL** and **LOG backups**.
 - Restore them with **NORECOVERY**.
 - Once ready to cut over, take final **tail-log backup**, restore with **RECOVERY**.

Downtime: Minutes to a few hours (mainly final log restore + application switchover).

Best for: Medium-low bandwidth, but allows controlled cutover.

Option B – Log Shipping

Process:

1. Take full backup, restore on EC2 with **NORECOVERY**.
2. Configure **log shipping** to continuously send T-Log backups from on-prem to EC2.
3. When ready, stop application writes, send final log backup, restore, bring DB online.

Downtime: Minutes.

Best for: Large DBs (1 TB+), where you can ship logs continuously until cutover.

Option C – Database Mirroring / Always On AG (Hybrid)

Process:

1. Create a site-to-site VPN or AWS Direct Connect.
2. Configure **Database Mirroring** or **Always On Availability Groups** between on-prem SQL Server and EC2 SQL Server.
3. Let it sync until ready, then perform a **failover**.

Downtime: Seconds to minutes.

Best for: Mission-critical migrations with almost zero downtime.

Note: Requires Enterprise Edition for AG; Mirroring works in Standard.

2. Migration to AWS RDS for SQL Server

RDS is managed by AWS, so no direct file access – some methods change.

Option A – Native Backup/Restore to S3

Process:

1. Use RDS option group to enable **SQLSERVER_BACKUP_RESTORE**.
2. Backup your on-prem DB to **.bak** and upload to S3.
3. Run **rds_restore_database** to restore it into RDS.

4. For minimal downtime:

- Restore full backup.
- Use transaction log backups via S3 until cutover.

Downtime: Minutes to hours depending on final log restore.

Best for: Simple, native migration with minimal tooling.

Option B – AWS DMS (Database Migration Service)

Process:

1. Create RDS instance.
2. Configure AWS DMS for **full load + ongoing replication** from on-prem SQL Server.
3. Once synced, stop writes on source, wait for final sync, then point apps to RDS.

Downtime: Minutes.

Best for: Heterogeneous migrations or when downtime must be near-zero.

Note: DMS works table-by-table – might need schema pre-creation for certain features.

Option C – Transactional Replication

Process:

1. Configure on-prem SQL Server as **Publisher** and RDS as **Subscriber**.
2. Snapshot initial data to RDS.
3. Let replication sync changes.
4. Cut over by disabling writes on-prem, wait for sync, switch apps.

Downtime: Seconds to minutes.

Best for: Near-zero downtime migrations.

Note: RDS supports being a subscriber, not a publisher.

3. Low-Bandwidth / High Data Volume Special Case

For 1 TB+ DBs over slow links:

- **AWS Snowball Edge:**
 - Export backup to Snowball appliance shipped by AWS.
 - AWS loads it into S3.
 - Restore into EC2 or RDS.
 - Ship only differential/log backups over network for cutover.

This avoids sending the full 1 TB over the wire.

Summary Table

Method	Target	Downtime	Complexity	Notes
Backup + Diff/Log Restore	EC2/RDS	Minutes–Hours	Low	Easy, works offline with S3/Snowball
Log Shipping	EC2	Minutes	Medium	Reliable for large DBs
Always On / Mirroring	EC2	Seconds	High	Requires VPN/Direct Connect
Native Backup/Restore to S3	RDS	Minutes–Hours	Low	Native RDS feature
AWS DMS	RDS	Minutes	Medium	Ongoing replication
Transactional Replication	RDS	Seconds–Minutes	Medium	RDS only as subscriber